

OOP

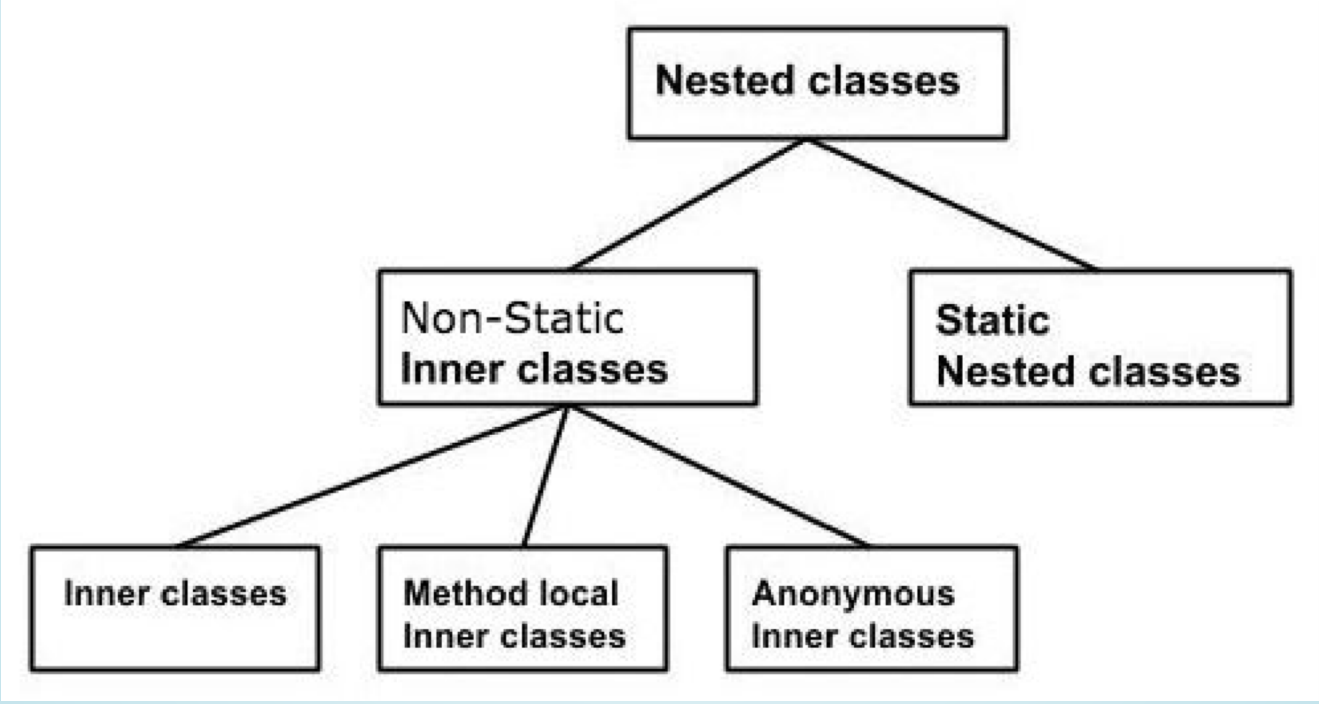
programare orientată pe obiecte

CURS06: CLASE INTERNE

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Universitatea Politehnica București

O clasa interna (Nested Class) se defineste in interiorul unei alte clase, denumita clasa externa (Outer Class), in diferite forme (ca membru, in interiorul unei metode, etc.).



Java Inner Classes tutorial

INNER CLASSES

alexandru.olteanu@upb.ro

Poate fi vizibila in exterior, caz in care poate fi accesata prin intermediul unei instante a clasei externe (ori ca rezultat al unei metode, ori prin instantiere)

```
public class Person {
    public class Account {
        public String bank ;
        public String iban ;
    }
    public Account getAccount () {
        Account c = new Account (" SwissBank ", " CH23 ...")
        return c;
    }
}
public class Main {
    public static void main ( String [] args ) {
        Person p1 = new Person () ;
        Person . Account c1 = p1. getAccount () ;
        Person . Account c2 = p1.new Account (" USBank ", "
    }
```

Putem sa accesam instanta clasei externe care a creat-o

```
public class Person {
    private String name;
    public class Account {
        public String bank ;
        public String iban ;
        public String toString() {
            return Person.this.name +"'s account : "+ iban ;
        }
    }
}
public class Main {
    public static void main ( String [] args ) {
        Person pl = new Person (" Mike ") ;
        Person . Account c = pl.new Account(" USBank ", " US23 .
        System.out.println(c) ; // ce se afiseaza ?
    }
```

Poate fi 'private', caz in care implementarea este ascunsa, dar interfata poate fi publica

```
interface BankAccount { }
public class Person {
    private class Account implements BankAccount {
        public String bank ;
        public String iban ;
    }
    public BankAccount getAccount () {
        Account c = new Account (" SwissBank ", " CH23 ...")
        return c;
    }
}
public class Main {
    public static void main ( String [] args ) {
        Person p = new Person () ;
        BankAccount c3 = p.getAccount () ; // ok
    }
```


alexandru.olteanu@upb.ro

METHOD LOCAL INNER CLASSES

alexandru.olteanu@upb.ro

Vizibilitatea clasei se limiteaza la metoda (desi codul binar se creeaza oricum). Pentru a putea folosi variabilele si parametrii metodei, clasa trebuie sa fie final.

```
public class TestMethodLocalInnerClass {
    public static void main ( String [] args ) {
        class Greeter implements Runnable {
            private final String _greeted ;
            public Greeter ( String greeted ) {
                super () ;
                _greeted = greeted ;
            }
            public void run () {
                System.out.println(" Hello %s!\n", _greeted
            }
        }
        new Greeter (" world ").run() ;
        new Greeter ("dog").run() ;
    }
}
```

ANONYMOUS INNER CLASSES

alexandru.olteanu@upb.ro

Daca o clasa este instantiata intr-un singur loc si apoi folosita (prin upcasting) doar prin intermediul clasei de baza sau al interfetei, numele sau nu mai este important.

```
public class TestAnonymousInnerClass {
    public static void main ( String [] args ) {
        new Runnable () {
            @Override
            public void run() {
                System.out.println(" Hello world!\n");
            }
        }.run() ;
    }
}
```


alexandru.olteanu@upb.ro

Exemple celebre: multi-threading

```
public static void main(String[] args) {
    new Thread ( new Runnable() {
        @Override
        public void run() {
            // do magic on separate thread
        }
    }).start () ;
}
```

Exemple celebre: event listeners

```
public class MyClass extends Applet {
    someObject.addMouseListener (new MouseAdapter () {
        public void mouseClicked (MouseEvent e) {
            // do magic when mouse is clicked
        }
    }) ;
}
```

LAMBDA EXPRESSIONS (QUICK INTRO)

alexandru.olteanu@upb.ro

Expresii lambda

```
parameter -> expression
(parameter1, parameter2) -> expression
(parameter1, parameter2) -> { code block }
```

```
n -> n % 2 != 0;
(char c) -> c == 'y';
(x, y) -> x + y;
(int a, int b) -> a * a + b * b;
() -> 42
() -> { return 3.14 };
(String s) -> { System.out.println(s); };
() -> { System.out.println("Hello World!"); };
```


alexandru.olteanu@upb.ro

Java nu suporta first class functions, asa ca expresiile lambda vor fi reprezentate sub forma unor obiecte atasate unor interfete functionale.

Interfata functionala (functional interface): interfata ce contine exact o metoda abstracta. Poate avea oricate metode default sau static, dar una singura sa fie abstracta.

```
@FunctionalInterface
interface Sayable{
    void say(String msg);
}
```

```
public static void main(String[] args) {
    Sayable greet = (m) -> "Hello "+m;
```

```
    System.out.println(greet.say("Mihai"));
}
```

Clase interne

ale

Expresii lambda: utile pentru a exprima functionalitate printr-o sintaxa comprimata

```
Map<Integer, String> nume = new HashMap<>();
nume.put(1, "Andreea");
nume.put(2, "Maria");
nume.put(3, "Mihai");
nume.forEach((key, value) -> System.out.println(key + " " + valu
```


alexandru.olteanu@upb.ro

Când veți scrie o clasă anonimă cu o singura metoda într-un IDE cum e IntelliJ, veți primi un warning care vă recomandă să o transformați într-o expresie lambda

```
public class TestAnonymousInnerClass {
    public static void main ( String [] args ) {
        new Runnable () {
            @Override
            public void run() {
                System.out.println(" Hello world!\n");
            }
        }.run() ;
    }
}
```

```
public class TestAnonymousInnerClass {
    public static void main ( String [] args ) {
        ((Runnable) () -> System.out.println(" Hello world!\n"))
```

```
    }
}
```

Clase interne

ale

Conditii clase anonime:

- O clasa anonima poate sa extinda o singura clasa sau sa implementeze o singura interfata
- O clasa anonima nu are constructori (nu are nume, nu am sti cum se cheama constructorul); in mod implicit se invoca constructorul din clasa de baza care se potriveste cel mai bine cu parametrii folositi la initializare (sau constructorul default al clasei de baza daca nu sunt folositi parametrii)

alexandru.olteanu@upb.ro

STATIC NESTED CLASSES

alexandru.olteanu@upb.ro

O clasa interna poate fi un membru static al clasei externe:

e.g. Map.Entry, java.awt.geom.Rectangle2D.Double

```
for (Map.Entry<Integer, String> entry : nume.entrySet())
{
  Integer key = entry.getKey();
  String value = entry.getValue();
}
```

```
Rectangle2D rect = new Rectangle2D.Double(leftX, topY, width, he
```


alexandru.olteanu@upb.ro

O clasa interna poate fi un membru static al clasei externe:

- nu va avea acces la instantele clasei externe (Outer.this), ci functioneaza ca o alta clasa top-level
- are sens sa fie interna pentru ca este folosita doar prin clasa externa (packaging convenience)
- nu avem nevoie de o instanta a clasei externe pentru a creea o instanta a clasei interne

Oracle docs